

Technical Reference Manual

AI Checkout Copilot Architecture & Implementation Spec

Aryaman Verma

Computer Science & Engineering Kalinga Institute of Industrial Technology

Contents

- 1 Architecture Overview 2
 - 1.1 The Web Worker Sandbox (Frontend) 2
 - 1.2 The Inference Proxy (Backend) 2
- 2 State Management & Data Parsing 2
 - 2.1 The “Wide Net” Reactive Data Parser 2
 - 2.2 Memory Caching & Rate Limiting 3
- 3 LLM Integration & Prompt Engineering 3
 - 3.1 Google Gemini 2.5 Flash 3
 - 3.2 System Prompt Architecture 3
- 4 Deployment & Security 4

1 Architecture Overview

The AI Checkout Copilot is a decoupled, serverless-ready application designed to operate within the strict security boundaries of the Shopify Checkout environment. The architecture is split between a sandboxed UI Extension and an external inference proxy.

1.1 The Web Worker Sandbox (Frontend)

Shopify Checkout UI Extensions do not run in a standard browser DOM. They execute inside a highly restricted Web Worker sandbox.

- **Constraint:** Direct DOM manipulation (`document.getElementById`), the `window` object, and standard React rendering are prohibited.
- **Solution:** The frontend is built using Preact and Shopify’s remote rendering UI components (`@shopify/ui-extensions/preact`). UI state is managed via signals and hook-based subscriptions rather than standard event listeners.

1.2 The Inference Proxy (Backend)

Because the Web Worker sandbox cannot securely store API keys and restricts certain outbound protocols, a Remix backend acts as a secure intermediary.

- **Responsibilities:** CORS handling, payload validation, prompt hydration, and secure communication with the Google Gemini SDK.
- **Routing:** During development, the proxy is exposed via Cloudflare Tunnels to provide a secure HTTPS endpoint that Shopify’s checkout environment accepts.

2 State Management & Data Parsing

2.1 The “Wide Net” Reactive Data Parser

Shopify’s Checkout API relies on reactive states that shift structurally depending on the specific API version and item schema (e.g., standard products vs. variants). Relying on static object paths results in fatal `undefined` errors.

To guarantee stability, the application employs a “Wide Net” traversal algorithm. It utilizes logical fallbacks to search for state across both `.current` (signals) and `.value` (static refs) paths simultaneously.

```
// Safely extracting total costs across API versions
const totalAmount =
  shopify.cost?.totalAmount?.current?.amount ||
  shopify.cost?.totalAmount?.value?.amount ||
  shopify.cart?.cost?.totalAmount?.value ||
  0;

// Deep schema traversal for merchandise titles
const lines = shopify.lines?.current || shopify.cart?.lines?.value || [];
const itemNames = lines
  .map((line) => line.title || line.merchandise?.title ||
```

```
line.merchandise?.product?.title)
  .filter(Boolean)
  .join(", ");
```

2.2 Memory Caching & Rate Limiting

Shopify's API triggers the `deliveryGroups.subscribe` listener aggressively (e.g., on keypresses in address fields, or when shipping options render). Passing every trigger to the LLM would result in HTTP 429 Rate Limit errors and high latency.

Implementation: A localized memory cache using Preact's `useRef` stores a stringified snapshot of the current variables.

```
const currentSnapshot = JSON.stringify({ city, totalAmount, itemNames,
shippingOptions });

if (lastRequestRef.current === currentSnapshot) return;
lastRequestRef.current = currentSnapshot; // Update cache and proceed to fetch
```

This guarantees the backend is only pinged when the financial or physical parameters of the cart actually mutate.

3 LLM Integration & Prompt Engineering

3.1 Google Gemini 2.5 Flash

The application leverages Google's `gemini-2.5-flash` model for its low-latency inference, which is critical for synchronous checkout flows.

3.2 System Prompt Architecture

The prompt is constructed dynamically on the backend, hydrating a strict instruction set with the JSON payload received from the frontend.

Key Directives:

1. **Data Grounding:** The prompt explicitly injects the user's `$${{subtotalAmount}}` and `${itemNames}`.
2. **Bundle Hallucination Control:** By providing the exact cart contents, the LLM is constrained to generating logical upsell relationships (e.g., bindings for snowboards) rather than generic cross-sells.
3. **Conversational Termination:** The prompt includes the strict rule: `"NEVER ask a question."` This prevents the LLM from attempting to initiate a chatbot-style dialogue, forcing it to act purely as an analytical engine.

4 Deployment & Security

- **CORS:** The backend explicitly handles `OPTIONS` preflight requests, returning `Access-Control-Allow-Origin: *` to prevent the browser from blocking the cross-origin fetch from the Shopify checkout domain.
- **Secret Management:** The `GEMINI_API_KEY` is completely isolated from the client bundle, injected exclusively into the Remix server context via `.env` variables or Cloudflare environment bindings.